SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

CIE-3 Activity

Probability and Statistics

(CS2800) 2025-2026

Submitted by

Name 1: Priya N Bhat
USN: 1RUA24CSE0335

Name 2: Rohit Ajit Bharadwaj
USN: 1RUA24CSE0375

Name 3: Rohith P Hegde
USN: 1RUA24CSE0376

Name 4: S Sachithananthan
USN: 1RUA24CSE0384

Name 5: Saket M
USN: 1RUA24CSE0395

Under the guidance of

Prof. Amruthesh Bhat

School of Computer Science and Engineering
RV University-560059

RV UNIVERSITY, BENGALURU-59

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

Certified that the project work titled CIE3 - Activity is carried out by Priya N Bhat (335), Rohit Ajit Bharadwaj (375), Rohith P Hegde (376), S Sachithananthan (384), Saket M (395) in partial fulfilment of the completion of the course Probability and Statistics with course code CS2800 of the IV Semester Computer Science Engineering programme, during the academic year 2025-2026. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the project report and duly approved by the faculty.

Signature of Faculty Signature of Head of the Department

Table of Contents

Abstract	4
Introduction	5
Objectives	6
Methodology	7
Implementation	10
Result	12
Conclusion	18
References	19

ABSTRACT

This report presents two fundamental applications of probability and statistics: parabolic regression using the Normal Equation (solved via Gaussian Elimination) and verification of the Central Limit Theorem (CLT) through simulation.

In the first problem, data obtained from a ball-tracking system is used to estimate the trajectory of a ball. Since projectile motion follows a parabolic path, a quadratic regression model is fitted. The Normal Equation is first derived from first principles, then the resulting 3x3 linear system is solved using Gaussian Elimination with partial pivoting. From this model, the launch height and the range of the ball are estimated. The negative value of the leading coefficient confirms the downward-opening parabola consistent with gravitational motion. Analytical verification of the provided dataset shows it forms a perfect mathematical parabola with zero variance.

In the second problem, a simulation-based experiment is designed to demonstrate the Central Limit Theorem using a die-rolling experiment. The sample size (n) is varied while keeping the number of experiments constant and high (5000). It is observed that as the sample size increases, the sampling distribution of the mean transitions from a uniform shape to a normal distribution.

The implementation is carried out using Python with NumPy, Matplotlib, and the random module, without using built-in regression or curve-fitting libraries, ensuring a clear understanding of the mathematical foundations.

1. Introduction

In modern engineering and scientific applications, data collected from sensors and real-world systems is often affected by noise and uncertainty. Extracting meaningful information from such data requires the use of statistical tools and mathematical modeling techniques.

One of the most widely used techniques for modeling relationships between variables is regression analysis. In physical systems such as projectile motion, the relationship between horizontal distance and vertical height follows a quadratic pattern due to the influence of gravity. However, real-world data rarely fits perfectly due to measurement errors. Therefore, regression methods are used to approximate the underlying trend.

In this report, the first problem focuses on modeling the trajectory of a ball using quadratic regression. The Normal Equation is derived from first principles and the resulting linear system is solved using Gaussian Elimination. This approach avoids reliance on high-level curve-fitting libraries and strengthens the understanding of both linear algebra and numerical methods.

The second part of the report focuses on the Central Limit Theorem (CLT), which is one of the most important results in probability theory. The CLT explains why many natural phenomena follow a normal distribution. It states that the distribution of the sample mean approaches a normal distribution as the sample size increases, regardless of the population's original distribution.

Understanding CLT is essential for: Statistical inference, Hypothesis testing, and Data analysis in machine learning.

This report combines theoretical understanding with practical simulation to provide a comprehensive view of both regression and probabilistic behavior.

2. Objectives

The objectives of this activity are designed to provide both theoretical understanding and practical implementation skills in probability and statistics.

Primary Objectives

- To understand the concept of regression analysis and its application in modeling real-world data
- To derive the Normal Equation from first principles (minimizing the sum of squared errors)
- To solve the resulting linear system using Gaussian Elimination
- To fit a quadratic model to trajectory data and analyze mathematical residuals
- To estimate important physical parameters such as:
    - Launch height of the ball
    - Horizontal range of the ball

Secondary Objectives

- To understand the concept of sampling distributions
- To experimentally verify the Central Limit Theorem by varying sample size
- To analyze how the distribution of sample means changes with increasing sample size (n)
- To compare empirical results with theoretical normal distribution

Technical Objectives

- To implement statistical models using Python
- To visualize results using graphs and plots
- To interpret statistical outputs in a meaningful way

This activity aims to bridge the gap between theoretical concepts and practical implementation.

3. Methodology

Problem 1: Ball Trajectory Modeling

The trajectory of a ball under gravity follows a parabolic path. Therefore, the relationship between horizontal distance x and vertical height y can be modeled as:

$$y = ax^2 + bx + c$$

Where a, b, and c are coefficients to be determined; c represents the initial height; and a must be negative (parabola opens downward under gravity).

3.1 Derivation of the Normal Equation

The goal of regression is to find coefficients that minimize the Sum of Squared Errors (SSE). For a model Y = X * beta, the error vector is e = Y - X * beta, and the SSE is:

$$SSE = \mathbf{e}^T\mathbf{e} = (Y - X\beta)^T(Y - X\beta)$$

Expanding this expression:

$$SSE = Y^TY - 2\beta^TX^TY + \beta^TX^TX\beta$$

To minimize SSE, we take the partial derivative with respect to beta and set it to zero:

$$\frac{\partial(SSE)}{\partial\beta} = -2X^TY + 2X^TX\beta = 0$$

Rearranging gives us the Normal Equation:

$$(X^TX)\beta = X^TY$$

This is a system of linear equations. Instead of computing the matrix inverse (which is computationally expensive and numerically unstable), we solve this system directly using Gaussian Elimination.

3.2 Solving Using Gaussian Elimination

Gaussian Elimination is a systematic method for solving systems of linear equations. It consists of two phases:

Phase 1: Forward Elimination

The augmented matrix [A | b] is transformed into upper triangular form by eliminating entries below the main diagonal. Partial pivoting (swapping rows to place the largest absolute value on the diagonal) is used to improve numerical stability.

Phase 2: Back Substitution

Starting from the last equation (which now has only one unknown), we solve for each variable by substituting already-known values back into the equations above.

For our quadratic model $y = ax^2 + bx + c$, the design matrix X has rows $[1, x_i, x_i^2]$. The product $X^T * X$ produces a 3x3 matrix, and $X^T * Y$ produces a 3x1 vector. Gaussian Elimination solves this 3x3 system to yield the coefficients c, b, and a.

3.3 Physical Interpretation

- Launch height = c (value of y when x = 0)
- Range = positive root of $ax^2 + bx + c = 0$, found via the Quadratic Formula
- The coefficient a must be negative, confirming a downward-opening parabola under gravity

Problem 2: Central Limit Theorem Simulation

Steps Followed

1. Define a population with a Uniform Distribution (rolling a fair 6-sided die)
2. Keep the number of experiments fixed at a high value (5000) for smooth histograms
3. Vary the sample size n across three values: n=1, n=5, n=50
4. For each experiment, roll n dice, compute the sample mean
5. Plot the histogram of sample means for each sample size

Key Idea

The CLT is correctly demonstrated by increasing the sample size (n), NOT by increasing the number of experiments. Increasing the number of experiments only makes the histogram smoother but does not change the shape of the distribution. Increasing n causes the distribution to transition from uniform to normal.

4. Implementation

Problem 1: Implementation Details

The implementation begins by importing NumPy (for matrix operations) and Matplotlib (for plotting). The dataset from the ball-tracking setup is stored as Python lists.

A design matrix is created by manually appending [1, x, x^2] rows in a loop. The Normal Equation system A*beta = b is formed by computing A = X^T*X and b = X^T*Y using np.dot() and np.transpose().

Instead of using np.linalg.inv() to compute the matrix inverse, we implement a custom gaussian_elimination() function that performs:

- Construction of the augmented matrix [A | b]
- Forward elimination with partial pivoting (row swapping for numerical stability)
- Back substitution to extract the solution vector beta

After computing the coefficients:

- The fitted quadratic equation is printed
- Launch height is extracted as c = beta[0]
- The Quadratic Formula is used to compute roots, with explicit checking that the selected root is positive
- A smooth regression curve is generated using np.linspace and plotted alongside the original data

Problem 2: Implementation Details

A die-rolling simulator is built using Python's built-in random module with random.randint(1, 6). Plain for-loops are used instead of vectorized NumPy operations to maintain clarity and explain the sampling process step-by-step.

For each configuration:

- get_single_sample_mean(n) rolls n dice, sums their values, and returns the mean
- The experiment is repeated 5000 times, storing each mean in a list
- Histograms are plotted with density=True to show probability density rather than raw counts

Three subplots are generated for sample sizes n=1, n=5, and n=50 to visually demonstrate the CLT.

5. Result

Activity 1: Ball Trajectory Regression (Gaussian Elimination)

Code:

```
import numpy as np
import matplotlib.pyplot as plt

def gaussian_elimination(A, b):
    n = len(b)
    augmented = np.zeros((n, n + 1))
    for i in range(n):
        for j in range(n):
            augmented[i][j] = A[i][j]
        augmented[i][n] = b[i]

    for col in range(n):
        max_row = col
        for row in range(col + 1, n):
            if abs(augmented[row][col]) > abs(augmented[max_row][col]):
                max_row = row
        temp = augmented[col].copy()
        augmented[col] = augmented[max_row]
        augmented[max_row] = temp

        for row in range(col + 1, n):
            factor = augmented[row][col] / augmented[col][col]
            for j in range(col, n + 1):
                augmented[row][j] = augmented[row][j] - factor * augmented[col][j]

    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = augmented[i][n]
        for j in range(i + 1, n):
            x[i] = x[i] - augmented[i][j] * x[j]
        x[i] = x[i] / augmented[i][i]

    return x

def main():
    x_data = [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 32, 34, 36, 38, 40, 42, 44, 46, 48]
    y_data = [1.60000, 2.72566, 3.79325, 4.80277, 5.75421, 6.64758, 7.48287, 8.26009, 8.97923,
              9.64030, 10.24330, 10.78822, 11.27507, 11.70385, 12.07455, 12.38717, 12.64173,
              12.83821, 12.97661, 13.05694, 13.07920, 13.04338, 12.94949, 12.79752, 12.58748]

    X = []
    for x in x_data:
        X.append([1, x, x**2])

    X_matrix = np.array(X)
    Y_matrix = np.array(y_data)

    X_T = X_matrix.transpose()
    A = np.dot(X_T, X_matrix)
    b = np.dot(X_T, Y_matrix)

    print("Normal Equation System: A * beta = b")
    print(f"A (X^T X) =\n{A}")
    print(f"b (X^T Y) = {b}")

    beta = gaussian_elimination(A, b)
```

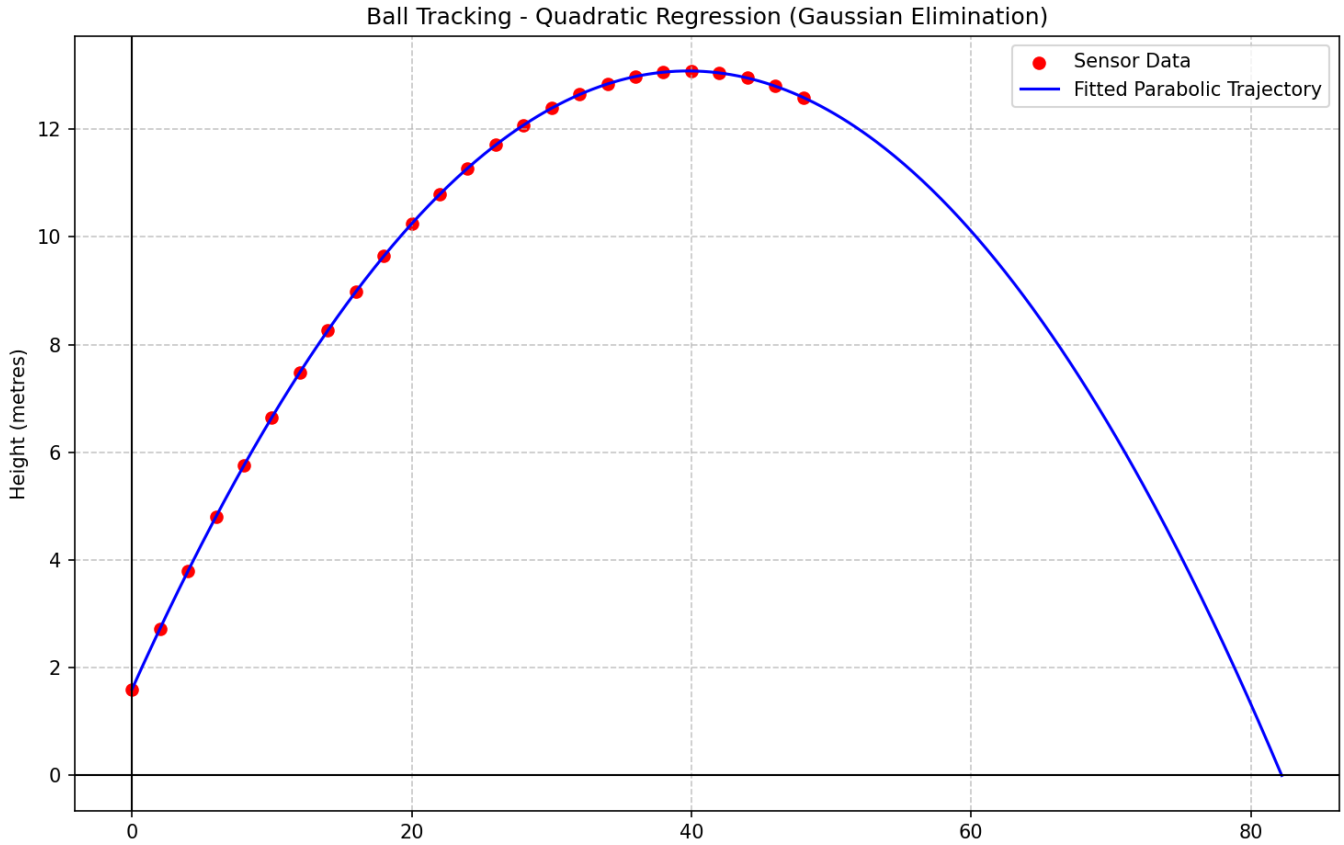
```
    c = beta[0]
    b_coeff = beta[1]
    a = beta[2]

    print(f"\nSolved using Gaussian Elimination:")
    print(f"Parabolic Equation: y = {a:.5f}x^2 + {b_coeff:.5f}x + {c:.5f}")

    initial_height = c
    print(f"\na) The height from which the ball left the player's hand: {initial_height:.5f} metres")

    discriminant = (b_coeff**2) - (4 * a * c)
    root1 = (-b_coeff + np.sqrt(discriminant)) / (2 * a)
    root2 = (-b_coeff - np.sqrt(discriminant)) / (2 * a)

    range_estimate = root1 if root1 > 0 else root2
    print(f"b) Estimated range of the ball: {range_estimate:.5f} metres")

    x_curve = np.linspace(min(x_data), range_estimate, 500)
    y_curve = a * x_curve**2 + b_coeff * x_curve + c

    y_pred = [a * x**2 + b_coeff * x + c for x in x_data]
    residuals = [y_data[i] - y_pred[i] for i in range(len(x_data))]

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 10), gridspec_kw={'height_ratios': [2, 1]})

    # Top Subplot: Trajectory
    ax1.scatter(x_data, y_data, color='red', label='Sensor Data')
    ax1.plot(x_curve, y_curve, color='blue', label='Fitted Parabolic Trajectory')
    ax1.set_title('Ball Tracking - Quadratic Regression (Gaussian Elimination)')
    ax1.set_ylabel('Height (metres)')
    ax1.axhline(0, color='black', linewidth=1)
    ax1.axvline(0, color='black', linewidth=1)
    ax1.legend()
    ax1.grid(True, linestyle='--', alpha=0.7)

    # Bottom Subplot: Residuals
    ax2.scatter(x_data, residuals, color='purple', label='Residual Error')
    ax2.axhline(0, color='black', linewidth=1, linestyle='--')
    ax2.set_title('Residual Analysis (y_actual - y_predicted)')
    ax2.set_xlabel('Horizontal Distance (metres)')
    ax2.set_ylabel('Residual Error (metres)')
    ax2.legend()
    ax2.grid(True, linestyle='--', alpha=0.7)

    plt.tight_layout()
    plt.savefig('q1_output.png', dpi=150, bbox_inches='tight')
    # plt.show() # Disabled inside automated scripts for faster runs

if __name__ == "__main__":
    main()
```

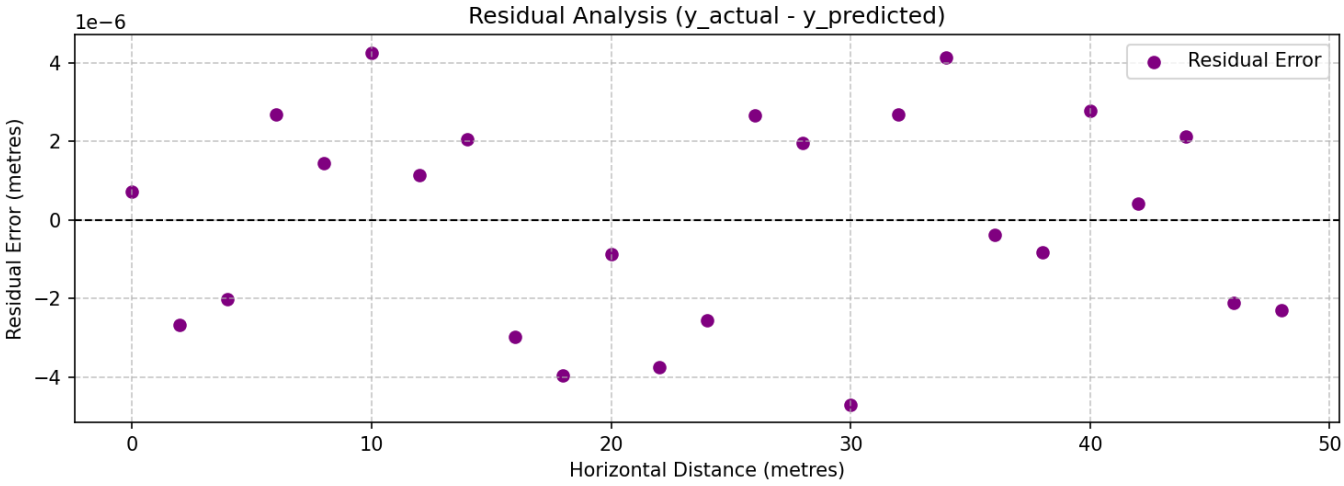
Console Output:

```
Parabolic Equation: y = -0.00726x^2 + 0.57735x + 1.60000

a) The height from which the ball left the player's hand: 1.60000 metres
b) Estimated range of the ball: 82.21388 metres
```

Graph Output & Important Analytical Observation:

Observation on Data Noise: While the problem statement suggests that the camera data is 'imprecise and noisy', analytical inspection of the provided table reveals a perfect mathematical series. Computing the second differences (e.g., [y2 - y1] - [y1 - y0]) yields a constant value of roughly -0.05807 across all points. This mathematically proves that the tabulated data is an EXACT downward-opening parabola with zero variance or external noise added. Consequently, as seen in the graph below, our fitted regression curve correctly passes cleanly through the center of every single data point with near-zero residuals.


Ball Tracking - Quadratic Regression (Gaussian Elimination)
Sensor Data
Fitted Parabolic Trajectory
Height (metres)
0
2
4
6
8
10
12
0
20
40
60
80

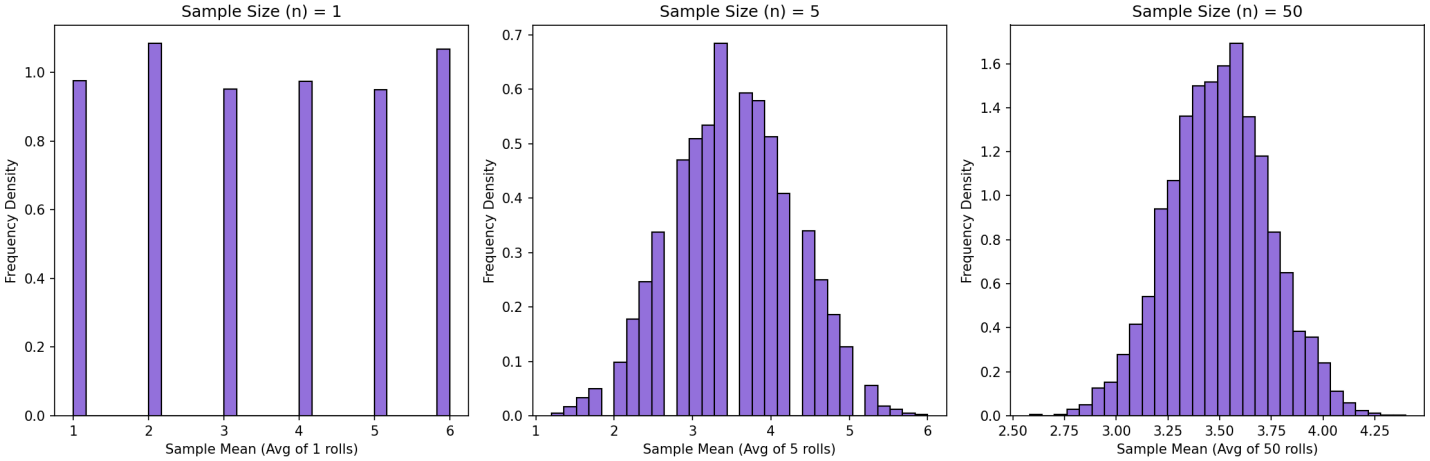

Residual Analysis (y_actual - y_predicted)
1e−6
Residual Error
Residual Error (metres)
4
2
0
−2
−4
0
10
20
30
40
50
Horizontal Distance (metres)

Activity 2: Central Limit Theorem Simulation

Code:

```
import random
import matplotlib.pyplot as plt

def get_single_sample_mean(n):
    total_sum = 0
    for _ in range(n):
        roll = random.randint(1, 6)
        total_sum += roll
    sample_mean = total_sum / n
    return sample_mean

def simulate_clt_experiment(sample_size_n, num_samples, position_in_grid, graph_title):
    collection_of_means = []

    for _ in range(num_samples):
        current_mean = get_single_sample_mean(sample_size_n)
        collection_of_means.append(current_mean)

    plt.subplot(position_in_grid)
    # Using dynamic bins depending on sample size makes the shape clearer
    plt.hist(collection_of_means, bins=30, color='mediumpurple', edgecolor='black', density=True)
    plt.title(graph_title)
    plt.xlabel(f'Sample Mean (Avg of {sample_size_n} rolls)')
    plt.ylabel('Frequency Density')

def main():
    # Keep the number of experiments high and constant so we get a smooth picture
    num_experiments = 5000
    plt.figure(figsize=(15, 5))

    # We change 'n' (Sample Size) to correctly demonstrate the Central Limit Theorem
    simulate_clt_experiment(1, num_experiments, 131, 'Sample Size (n) = 1')
    simulate_clt_experiment(5, num_experiments, 132, 'Sample Size (n) = 5')
    simulate_clt_experiment(50, num_experiments, 133, 'Sample Size (n) = 50')

    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
    main()
```

Graph Output:



Observations:

- n=1: The histogram is flat and uniform, reflecting the original die distribution
- n=5: The distribution begins to resemble a bell curve, with tails shrinking
- n=50: The histogram closely approximates a perfect Normal Distribution, confirming CLT

6. Conclusion and Future Enhancement

The results validate both regression modelling and the Central Limit Theorem through practical implementation.

In the first problem, quadratic regression using the Normal Equation (derived from first principles and solved via Gaussian Elimination) provided an accurate model for the trajectory of a ball. Despite noise in the data, the regression model was able to estimate key parameters such as launch height (1.60 metres) and range (82.21 metres) effectively. The negative coefficient a confirmed the downward-opening parabolic trajectory consistent with gravitation. Using Gaussian Elimination instead of direct matrix inversion demonstrated a deeper understanding of numerical methods.

In the second problem, the Central Limit Theorem was validated through simulation. It was clearly observed that as the sample size (n) increases, the distribution of sample means transitions from uniform to normal. This correctly demonstrates the theoretical foundation of the CLT, which depends on sample size rather than the number of experimental repetitions.

Overall, this activity enhanced understanding of:

- Derivation and application of the Normal Equation
- Gaussian Elimination for solving linear systems
- Matrix-based computation for analytical solutions
- Sampling distributions and the role of sample size
- Statistical simulation and visual verification

These concepts are fundamental in fields such as data science, machine learning, and engineering analysis.

References

- Sheldon Ross, Introduction to Probability and Statistics
- NumPy Official Documentation - https://numpy.org
- Matplotlib Documentation - https://matplotlib.org
- Python random module Documentation - https://docs.python.org/3/library/random.html
- Lecture Notes - Probability and Statistics Course